



Advanced JPA Hibernate

Topics

- Basics are already covered in Last Post
- Spring boot Integration with JPA and Database
- Entity Mapping and keys
- JPA Auditing with JWT Security
- Pagination and Sorting
- Criteria Builder
- Transaction Management
- Catching strategies with JPA



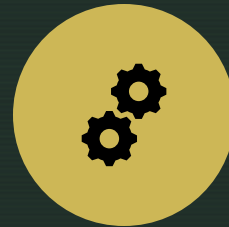
Spring Boot Integration with JPA and Database



INTRODUCTION
TO SPRING
BOOT AND JPA



SETTING UP
SPRING BOOT
WITH JPA



CONFIGURING
DATA SOURCE



CRUD
OPERATIONS
WITH JPA



EXCEPTION
HANDLING

1.Introduction to Spring Boot and JPA



Spring Boot: A framework that simplifies the setup and development of Spring applications. It provides defaults for code and annotation configuration to reduce boilerplate code.



JPA (Java Persistence API): A specification for accessing, persisting, and managing data between Java objects and relational databases. It provides a standard way to map Java objects to database tables.

2 and 3 Setting up Spring Boot with JPA

- Dependencies: Add spring-boot-starter-data-jpa and spring-boot-starter-web to your pom.xml or build.gradle.
- Application Properties: Configure database connection settings in application.properties or application.yml.
- DataSource Bean: Define a DataSource bean if you need custom configuration.

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/yourdb
2 spring.datasource.username=root
3 spring.datasource.password=yourpassword
4 spring.jpa.hibernate.ddl-auto=update
5
```

```
1 @Bean
2 public DataSource dataSource() {
3     return DataSourceBuilder.create()
4         .url("jdbc:mysql://localhost:3306/yourdb")
5         .username("root")
6         .password("yourpassword")
7         .build();
8 }
9
```

4.CRUD Operations with JPA

- Entity Class: Define your entity class with `@Entity` and `@Id` annotations.
- Repository Interface: Create a repository interface extending `JpaRepository`.

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    // getters and setters
}
```

```
1 public interface UserRepository extends JpaRepository<User, Long> {
2 }
3
```

4.CRUD Operations with JPA conti...

- Service Layer: Implement CRUD operations in the service layer

```
1  @Service
2  public class UserService {
3      @Autowired
4      private UserRepository userRepository;
5
6      public User saveUser(User user) {
7          return userRepository.save(user);
8      }
9
10     public List<User> getAllUsers() {
11         return userRepository.findAll();
12     }
13
14     public Optional<User> getUserById(Long id) {
15         return userRepository.findById(id);
16     }
17
18     public void deleteUser(Long id) {
19         userRepository.deleteById(id);
20     }
21 }
22
```

```
1 @ControllerAdvice
2 public class GlobalExceptionHandler {
3     @ExceptionHandler(ResourceNotFoundException.class)
4     public ResponseEntity<?>
5     handleResourceNotFoundException(ResourceNotFoundException ex) {
6         return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
7     }
8 }
```

Global Exception Handler: Use @ControllerAdvice to handle exceptions globally.

5. Exception Handling

2.Entity Mapping and Keys

- 1.Entity Mapping Basics
- 2.Primary Key Configuration
- 3.Foreign Key Relationships
- 4.Composite Key Mapping
- 5.Best Practices for Entity Mapping

1.Entity Mapping

@Entity

```
public class User {
```

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private String name;

 private String email;

 // getters and setters

■ }

2. Primary Key Configuration

```
1 @Id
2 @GeneratedValue(strategy = GenerationType.IDENTITY)
3 private Long id;
4
```

- Id Annotation: Use @Id to define the primary key.
- GeneratedValue Annotation: Use @GeneratedValue to specify how the primary key should be generated.

3. Foreign Key Relationships

- One-to-One Relationship:
Use `@OneToOne` and `@JoinColumn` to define a one-to-one relationship.
- One-to-Many Relationship:
Use `@OneToMany` and `@JoinColumn` to define a one-to-many relationship

```
1 @OneToOne
2 @JoinColumn(name = "profile_id")
3 private UserProfile profile;
4
```

```
1 @OneToMany(mappedBy = "user", fetch = FetchType.LAZY, cascade =
  CascadeType.ALL)
2 private List<Order> orders;
3
```

3. Foreign Key Relationships conti...

```
1 @ManyToOne
2 @JoinColumn(name = "role_id")
3 private Role role;
4
```

```
1 @ManyToMany
2 @JoinTable(
3     name = "user_group",
4     joinColumns = @JoinColumn(name = "user_id"),
5     inverseJoinColumns = @JoinColumn(name = "group_id")
6 )
7 private Set<Group> groups;
8
```

- Many-to-One Relationship:
Use `@ManyToOne` and `@JoinColumn` to define a many-to-one relationship.
- Many-to-Many Relationship:
Use `@ManyToMany` and `@JoinTable` to define a many-to-many relationship.

4. Composite Key Mapping

- EmbeddedId Annotation:
Use @EmbeddedId to define a composite key

```
1  @Embeddable
2  public class UserGroupId implements Serializable {
3      private Long userId;
4      private Long groupId;
5      // getters and setters
6  }
7
8  @Entity
9  public class UserGroup {
10     @EmbeddedId
11     private UserGroupId id;
12     // other fields and methods
13 }
14
```

5. Best Practices for Entity Mapping

- Use Lazy Loading: Prefer lazy loading for relationships to improve performance.
- avoid Eager Fetching: Use eager fetching sparingly as it can lead to performance issues.
- Define Proper Cascade Types: Use appropriate cascade types (`CascadeType.PERSIST`, `CascadeType.MERGE`, etc.) based on your use case.
- Use DTOs for Complex Queries: Use Data Transfer Objects (DTOs) to fetch and transfer data efficiently.
- Normalize Database Schema: Ensure your database schema is normalized to avoid redundancy and maintain data integrity.
- Use Indexes: Create indexes on columns that are frequently used in queries to improve performance.

JPA Auditing with JWT Security

1. Introduction to JPA Auditing
2. Setting up Auditing in JPA
3. Integrating JWT Security
4. Implementing Auditing with JWT



1. Introduction to JPA Auditing

- Purpose: JPA Auditing allows you to automatically capture and store metadata about entity changes, such as who created or modified an entity and when these changes occurred.
- Annotations: Key annotations include @CreatedBy, @LastModifiedBy, @CreatedDate, and @LastModifiedDate.

```
1  @Entity
2  public class User {
3      @Id
4      @GeneratedValue(strategy = GenerationType.IDENTITY)
5      private Long id;
6      private String name;
7      private String email;
8
9      @CreatedBy
10     private String createdBy;
11
12     @CreatedDate
13     private LocalDateTime createdDate;
14
15     @LastModifiedBy
16     private String lastModifiedBy;
17
18     @LastModifiedDate
19     private LocalDateTime lastModifiedDate;
20     // getters and setters
21 }
22
```

2. Setting up Auditing in JPA

- Enable Auditing:
Use `@EnableJpaAuditing` in your configuration class.
- AuditorAware Implementation:
Implement `AuditorAware` to provide the current auditor (user).

```
1  @Configuration
2  @EnableJpaAuditing
3  public class JpaConfig {
4  }
5
```

```
1  @Component
2  public class AuditorAwareImpl implements AuditorAware<String> {
3      @Override
4      public Optional<String> getCurrentAuditor() {
5          return Optional.of("current_user"); // Replace with actual user
6          retrieval logic
7      }
8  }
```

3. Integrating JWT Security

```
{  
  "header": {  
    "alg": "HS256",  
    "typ": "JWT"
```

```
  },
```

```
  "payload": {  
    "sub": "1234567890",  
    "name": "John Doe",  
    "iat": 1516239022
```

- JWT Basics: JSON Web Tokens (JWT) are used for securely transmitting information between parties as a JSON object. They are commonly used for authentication and authorization.

- JWT Structure: A JWT consists of three parts: Header, Payload, and Signature.

```
  },
```

```
  "signature": "SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_
```

```
}
```

```

1  @Component
2  public class AuditorAwareImpl implements AuditorAware<String> {
3      @Override
4      public Optional<String> getCurrentAuditor() {
5          Authentication authentication =
            SecurityContextHolder.getContext().getAuthentication();
6          if (authentication == null || !authentication.isAuthenticated()) {
7              return Optional.empty();
8          }
9          return Optional.of(((UserDetails)
            authentication.getPrincipal()).getUsername());
10     }
11 }
12

```

```

1  @Configuration
2  public class SecurityConfig extends WebSecurityConfigurerAdapter {
3      @Override
4      protected void configure(HttpSecurity http) throws Exception {
5          http.csrf().disable()
6              .authorizeRequests()
7              .antMatchers("/api/**").authenticated()
8              .and()
9              .addFilter(new JWTAuthenticationFilter(authenticationManager()))
10             .addFilter(new JWTAuthorizationFilter(authenticationManager()));
11     }
12 }
13

```

4. Implementing Auditing with JWT

- Extract User from JWT: Implement logic to extract the user information from the JWT token.
- Security Configuration: Configure Spring Security to use JWT for authentication

4. Pagination and Sorting

- Introduction to Pagination and Sorting
- Implementing Pagination in JPA
- Sorting Data with JPA
- Custom Pagination and Sorting
- Performance Considerations

1.Introduction to Pagination and Sorting

- Pagination and sorting are essential for handling large datasets efficiently. Pagination breaks down the dataset into manageable chunks, while sorting arranges the data in a specified order.

< Back 1 **2** 3 4 5 Next >

<< First < Back 1 **2** 3 4 ... 25 Next > Last >>

< Back 1 ... 98 99 100 **101** 102 103 ... 125 Next > Page 101 **Go**

110-120 of 1,250

< Back **1** 2 3 4 5 6 7 8 ... 25 Next > Result per page 50 ▾

1-50 of 1,250

Result per page 50 ▾ 1-50 of 1,250 << < > >>

2. Implementing Pagination in JPA

To implement pagination, we use the Pageable interface provided by Spring Data JPA.

Repository Interface

- Extend JpaRepository and PagingAndSortingRepository:

```
1 public interface UserRepository extends JpaRepository<User, Long>,
   PagingAndSortingRepository<User, Long> {
2 }
3
```

2. Implementing Pagination in JPA conti..

- Service Layer :
Implement pagination
logic in the service
layer:

```
1  @Service
2  public class UserService {
3      @Autowired
4      private UserRepository userRepository;
5
6      public Page<User> getUsers(int page, int size) {
7          Pageable pageable = PageRequest.of(page, size);
8          return userRepository.findAll(pageable);
9      }
10 }
11
```

2. Implementing Pagination in JPA conti..

Controller Layer

- Create an endpoint to handle pagination requests:

```
1 @RestController
2 @RequestMapping("/api/users")
3 public class UserController {
4     @Autowired
5     private UserService userService;
6
7     @GetMapping
8     public ResponseEntity<Page<User>> getUsers(
9         @RequestParam(defaultValue = "0") int page,
10        @RequestParam(defaultValue = "10") int size) {
11        Page<User> users = userService.getUsers(page, size);
12        return new ResponseEntity<>(users, HttpStatus.OK);
13    }
14 }
15
```

3. Sorting Data with JPA

Sorting can be combined with pagination by specifying the sort criteria in the PageRequest.

Service Layer

- Update the service layer to include sorting:

```
1 @Service
2 public class UserService {
3     @Autowired
4     private UserRepository userRepository;
5
6     public Page<User> getUsers(int page, int size, String sortBy) {
7         Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
8         return userRepository.findAll(pageable);
9     }
10 }
11
```

2. Implementing Pagination in JPA conti..

Controller Layer

- Create an endpoint to handle pagination and sorting requests:

```
1  @RestController
2  @RequestMapping("/api/users")
3  public class UserController {
4      @Autowired
5      private UserService userService;
6
7      @GetMapping
8      public ResponseEntity<Page<User>> getUsers(
9          @RequestParam(defaultValue = "0") int page,
10         @RequestParam(defaultValue = "10") int size,
11         @RequestParam(defaultValue = "id") String sortBy) {
12         Page<User> users = userService.getUsers(page, size, sortBy);
13         return new ResponseEntity<>(users, HttpStatus.OK);
14     }
15 }
16
```

4. Custom Pagination and Sorting

For more complex scenarios, you can customize pagination and sorting further.

Service Layer

- Include sorting direction (ascending or descending):

```
1  @Service
2  public class UserService {
3      @Autowired
4      private UserRepository userRepository;
5
6      public Page<User> getUsersWithSorting(int page, int size, String sortBy,
7      String direction) {
8          Sort sort = direction.equalsIgnoreCase(Sort.Direction.ASC.name()) ?
9          Sort.by(sortBy).ascending() : Sort.by(sortBy).descending();
10         Pageable pageable = PageRequest.of(page, size, sort);
11         return userRepository.findAll(pageable);
12     }
13 }
```

4. Custom Pagination and Sorting conti..

Controller Layer

Create an endpoint to handle custom pagination and sorting requests:

```
1 @RestController
2 @RequestMapping("/api/users")
3 public class UserController {
4     @Autowired
5     private UserService userService;
6
7     @GetMapping("/sorted")
8     public ResponseEntity<Page<User>> getUsersWithSorting(
9         @RequestParam(defaultValue = "0") int page,
10        @RequestParam(defaultValue = "10") int size,
11        @RequestParam(defaultValue = "id") String sortBy,
12        @RequestParam(defaultValue = "asc") String direction) {
13        Page<User> users = userService.getUsersWithSorting(page, size, sortBy,
14        direction);
15        return new ResponseEntity<>(users, HttpStatus.OK);
16    }
17 }
```



Indexing: Ensure that the columns used for sorting and filtering are indexed to improve query performance.



Page Size: Choose an appropriate page size to balance between performance and usability.



Caching: Implement caching strategies to reduce database load for frequently accessed data.



Lazy Loading: Use lazy loading for related entities to avoid fetching unnecessary data.

5. Performance Considerations



Criteria Builder

- Introduction to Criteria API
 - Building Queries with Criteria Builder
 - Dynamic Queries with Criteria API
 - Using Criteria Builder for Complex Queries
 - Best Practices for Criteria API
- 

1. Introduction to Criteria API

The Criteria API is part of JPA and allows you to build type-safe queries using Java objects. This approach is beneficial for creating dynamic queries and handling complex query requirements programmatically. 2. Building Queries with Criteria Builder

- To use the Criteria API, you need to create an instance of CriteriaBuilder and use it to construct your query. Example: Find Users by Name

```
1 import org.springframework.beans.factory.annotation.Autowired;
2 import org.springframework.stereotype.Service;
3 import javax.persistence.EntityManager;
4 import javax.persistence.criteria.CriteriaBuilder;
5 import javax.persistence.criteria.CriteriaQuery;
6 import javax.persistence.criteria.Root;
7 import java.util.List;
8
9 @Service
10 public class UserService {
11     @Autowired
12     private EntityManager entityManager;
13
14     public List<User> findUsersByName(String name) {
15         CriteriaBuilder cb = entityManager.getCriteriaBuilder();
16         CriteriaQuery<User> query = cb.createQuery(User.class);
17         Root<User> root = query.from(User.class);
18         query.select(root).where(cb.equal(root.get("name"), name));
19
20         return entityManager.createQuery(query).getResultList();
21     }
22 }
```

2. Dynamic Queries with Criteria API

The Criteria API allows you to build dynamic queries based on various conditions.

- Example: Find Users by Multiple Criteria

```
11 @Service
12 public class UserService {
13     @Autowired
14     private EntityManager entityManager;
15
16     public List<User> findUsersByCriteria(String name, Integer minAge, String
city) {
17         CriteriaBuilder cb = entityManager.getCriteriaBuilder();
18         CriteriaQuery<User> query = cb.createQuery(User.class);
19         Root<User> root = query.from(User.class);
20
21         List<Predicate> predicates = new ArrayList<>();
22         if (name != null) {
23             predicates.add(cb.equal(root.get("name"), name));
24         }
25         if (minAge != null) {
26             predicates.add(cb.greaterThan(root.get("age"), minAge));
27         }
28         if (city != null) {
29             predicates.add(cb.equal(root.get("city"), city));
30         }
31
32         query.select(root).where(cb.and(predicates.toArray(new Predicate[0])));
33
34         return entityManager.createQuery(query).getResultList();
35     }
36 }
```

3. Using Criteria Builder for Complex Queries

- To use the Criteria API, you need to create an instance of CriteriaBuilder and use it to construct your query.
- Example: Find Users by Name

```
9  @Service
10 public class UserService {
11     @Autowired
12     private EntityManager entityManager;
13
14     public List<User> findUsersByName(String name) {
15         CriteriaBuilder cb = entityManager.getCriteriaBuilder();
16         CriteriaQuery<User> query = cb.createQuery(User.class);
17         Root<User> root = query.from(User.class);
18         query.select(root).where(cb.equal(root.get("name"), name));
19
20         return entityManager.createQuery(query).getResultList();
21     }
22 }
23
```



Type Safety: Use the Criteria API to leverage type safety and avoid runtime errors.



Reuse Predicates: Reuse predicates to build complex queries incrementally.



Avoid Overcomplicating: Use the Criteria API for complex queries, but prefer JPQL for simpler queries to keep the code readable.



Performance Considerations: Optimize queries for performance by using appropriate indexes and avoiding unnecessary joins.

4. Best Practices for Criteria API



Transaction Management

- Introduction to Transaction Management
- Declarative Transaction Management
- Programmatic Transaction Management
- Handling Transaction Rollbacks

1. Introduction to Transaction Management

- Transaction management ensures that a series of operations either all succeed or all fail, maintaining data integrity. Spring Boot provides robust support for transaction management through the `@Transactional` annotation and the `PlatformTransactionManager`.
- Spring Boot simplifies transaction management configuration. You can configure transaction management in the `application.properties` file and use the `@Transactional` annotation to manage transactions.

2. Declarative Transaction Management

- Declarative transaction management uses annotations to manage transactions.

The `@Transactional` annotation can be applied at the class or method level.

- Example:
Using `@Transactional` Annotation

```
6 public class UserService {
7     @Autowired
8     private UserRepository userRepository;
9
10    @Transactional
11    public void createUser(User user) {
12        userRepository.save(user);
13    }
14
15    @Transactional(readOnly = true)
16    public User getUser(Long id) {
17        return userRepository.findById(id).orElse(null);
18    }
19 }
```

```

1  import org.springframework.beans.factory.annotation.Autowired;
2  import org.springframework.stereotype.Service;
3  import org.springframework.transaction.PlatformTransactionManager;
4  import org.springframework.transaction.TransactionDefinition;
5  import org.springframework.transaction.TransactionStatus;
6  import org.springframework.transaction.support.DefaultTransactionDefinition;
7
8  @Service
9  public class UserService {
10     @Autowired
11     private UserRepository userRepository;
12
13     @Autowired
14     private PlatformTransactionManager transactionManager;
15
16     public void createUser(User user) {
17         DefaultTransactionDefinition def = new DefaultTransactionDefinition();
18         def.setName("createUserTransaction");
19         def.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);
20
21         TransactionStatus status = transactionManager.getTransaction(def);
22         try {
23             userRepository.save(user);
24             transactionManager.commit(status);
25         } catch (Exception e) {
26             transactionManager.rollback(status);
27             throw e;
28         }
29     }
30 }

```

3. Programmatic Transaction Management :

- Programmatic transaction management involves managing transactions explicitly in your code using the PlatformTransactionManager.
- Example: Using PlatformTransactionManager

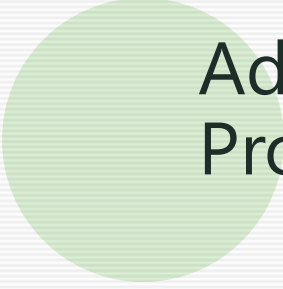
4. Handling Transaction Rollbacks

- Spring Boot allows you to handle transaction rollbacks declaratively using the `@Transactional` annotation or programmatically using the `PlatformTransactionManager`.
- Example: Declarative Rollback

```
1 import org.springframework.beans.factory.annotation.Autowired;
2 import org.springframework.stereotype.Service;
3 import org.springframework.transaction.annotation.Transactional;
4
5 @Service
6 public class UserService {
7     @Autowired
8     private UserRepository userRepository;
9
10    @Transactional(rollbackFor = Exception.class)
11    public void createUser(User user) throws Exception {
12        userRepository.save(user);
13        if (someCondition) {
14            throw new Exception("Rollback transaction");
15        }
16    }
17 }
18
```

Example: Programmatic Rollback

```
8  @Service
9  public class UserService {
10     @Autowired
11     private UserRepository userRepository;
12
13     @Autowired
14     private PlatformTransactionManager transactionManager;
15
16     public void createUser(User user) throws Exception {
17         DefaultTransactionDefinition def = new DefaultTransactionDefinition();
18         def.setName("createUserTransaction");
19         def.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);
20
21         TransactionStatus status = transactionManager.getTransaction(def);
22         try {
23             userRepository.save(user);
24             if (someCondition) {
25                 throw new Exception("Rollback transaction");
26             }
27             transactionManager.commit(status);
28         } catch (Exception e) {
29             transactionManager.rollback(status);
30             throw e;
31         }
32     }
33 }
34
```



Advanced Concepts: Isolation Levels and Propagation Behaviors

Isolation Levels :

Isolation levels define how transactions interact with each other and ensure data consistency. Spring supports several isolation levels:

- **DEFAULT:** Uses the default isolation level of the underlying database.
- **READ_UNCOMMITTED:** Allows dirty reads, meaning a transaction can read data that has been modified but not yet committed by other transactions.
- **READ_COMMITTED:** Prevents dirty reads; a transaction can only read data that has been committed.
- **REPEATABLE_READ:** Ensures that if a row is read twice in the same transaction, the result will always be the same.
- **SERIALIZABLE:** The highest isolation level, ensuring complete isolation from other transactions.
- **Example:** Using Isolation Levels

```
1  import org.springframework.beans.factory.annotation.Autowired;
2  import org.springframework.stereotype.Service;
3  import org.springframework.transaction.annotation.Transactional;
4
5  @Service
6  public class UserService {
7      @Autowired
8      private UserRepository userRepository;
9
10     @Transactional(isolation = Isolation.SERIALIZABLE)
11     public void createUser(User user) {
12         userRepository.save(user);
13     }
14 }
15
```

Example: Using Isolation Levels

Propagation Behaviors

Propagation behaviors define how transactions relate to each other.

Spring supports several propagation behaviors:

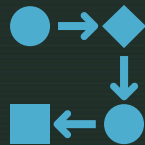
- **REQUIRED:** Uses the current transaction if one exists; otherwise, creates a new transaction.
- **REQUIRES_NEW:** Always creates a new transaction, suspending the current transaction if one exists.
- **SUPPORTS:** Uses the current transaction if one exists; otherwise, executes non-transactionally.
- **NOT_SUPPORTED:** Executes non-transactionally, suspending the current transaction if one exists.
- **MANDATORY:** Requires an existing transaction; throws an exception if none exists.
- **NEVER:** Executes non-transactionally; throws an exception if a transaction exists.
- **NESTED:** Executes within a nested transaction if a current transaction exists.

Example: Using Propagation Behaviors

```
1 import org.springframework.beans.factory.annotation.Autowired;
2 import org.springframework.stereotype.Service;
3 import org.springframework.transaction.annotation.Transactional;
4
5 @Service
6 public class UserService {
7     @Autowired
8     private UserRepository userRepository;
9
10    @Transactional(propagation = Propagation.REQUIRES_NEW)
11    public void createUser(User user) {
12        userRepository.save(user);
13    }
14
15    @Transactional(propagation = Propagation.REQUIRED)
16    public void updateUser(User user) {
17        userRepository.save(user);
18    }
19 }
20
```



Type Safety: Use the Criteria API to leverage type safety and avoid runtime errors.



Reuse Predicates: Reuse predicates to build complex queries incrementally.



Avoid Overcomplicating: Use the Criteria API for complex queries, but prefer JPQL for simpler queries to keep the code readable.



Performance Considerations: Optimize queries for performance by using appropriate indexes and avoiding unnecessary joins.

Best Practices for Transaction Management

▀ Caching Strategies with JPA

Introduction to Caching in JPA

First-Level Cache

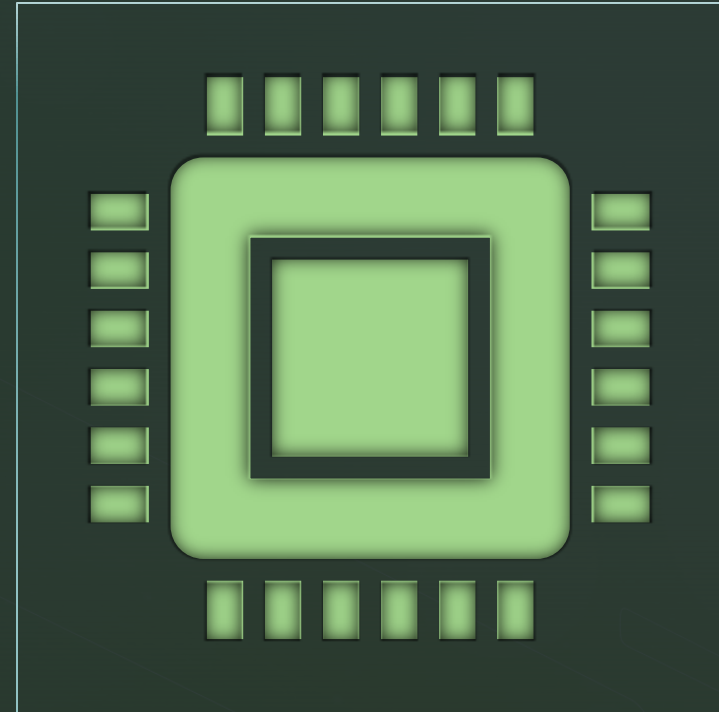
Second-Level Cache

Configuring Cache Providers

Best Practices for Caching

Introduction to Caching in JPA

- Caching in JPA improves performance by reducing database access. It involves storing frequently accessed data closer to the application. JPA supports two levels of caching: first-level and second-level caches.



Caching levels

First-Level Cache

The first-level cache, also known as the EntityManager cache, is session-scoped and ensures that each entity instance is loaded only once within the same session. This cache is automatically managed by the EntityManager and is useful for transaction-scoped caching.

Second-Level Cache

- The second-level cache is shared across sessions and is managed by the SessionFactory. It is useful for caching data that is frequently accessed by multiple sessions. Hibernate, a popular JPA implementation, supports second-level caching and can be configured to use various cache providers like EhCache, Infinispan, and Redis.

Redis Cache

Configuring Cache Providers

- To configure cache providers in JPA, you need to set up the cache provider and integrate it with your JPA implementation. For example, with Hibernate, you can use EhCache or Redis by adding the necessary dependencies and configuring the cache settings in your application properties.



redis

Configuring Redis in Spring Boot

First, add the necessary dependencies to your pom.xml

```
1 <dependency>
2     <groupId>org.springframework.boot</groupId>
3     <artifactId>spring-boot-starter-data-jpa</artifactId>
4 </dependency>
5 <dependency>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-data-redis</artifactId>
8 </dependency>
9 <dependency>
10    <groupId>org.springframework.boot</groupId>
11    <artifactId>spring-boot-starter-cache</artifactId>
12 </dependency>
13 <dependency>
14    <groupId>org.hibernate</groupId>
15    <artifactId>hibernate-ehcache</artifactId>
16 </dependency>
17
```

Configure Redis

3. Configure Redis: Add Redis dependencies to your project and configure Redis in your application properties.

```
1  spring.redis.host=localhost
2  spring.redis.port=6379
3  spring.cache.type=redis
4
```

Best Practices for Caching with Spring Boot 3.4 and Redis Cache

1. Enable Caching: Add the `@EnableCaching` annotation to your Spring Boot application configuration class to enable caching.

```
1  @Configuration
2  @EnableCaching
3  public class CacheConfig {
4  }
5
```

4. Create Redis Cache Manager:

Define a `RedisCacheManager` bean in your configuration class.

```
1 @Bean
2 public RedisCacheManager cacheManager(RedisConnectionFactory
3     connectionFactory) {
4     return RedisCacheManager.builder(connectionFactory)
5         .cacheDefaults(RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.
6             ofMinutes(10)))
7         .build();
8 }
```

Caching conti..

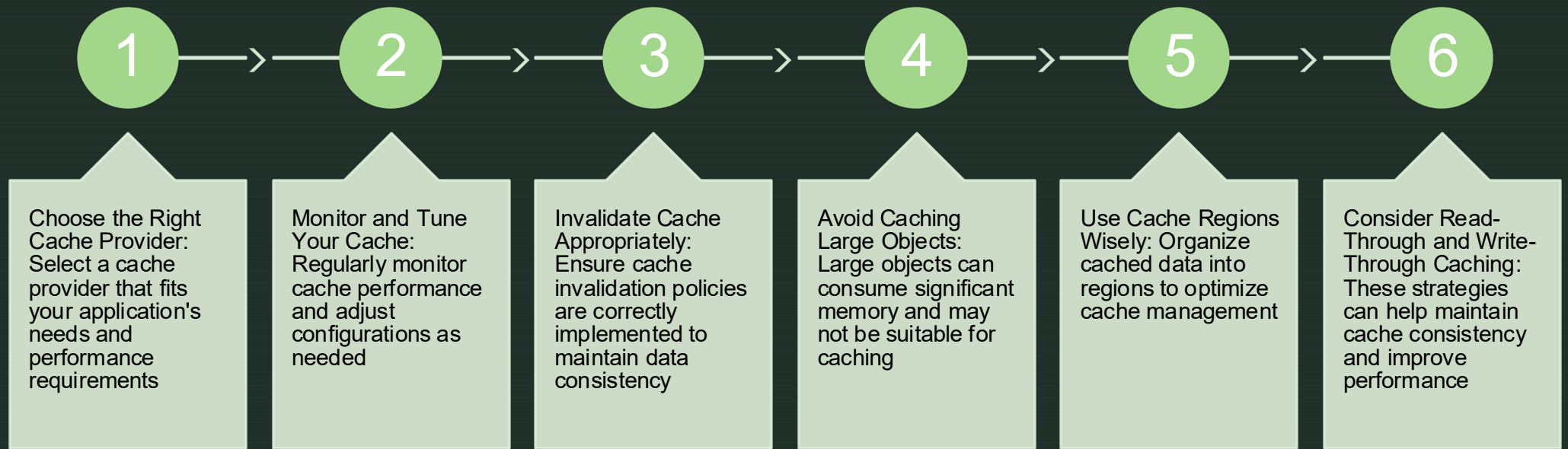
2. Use Cache Annotations: Utilize Spring's cache annotations like `@Cacheable`, `@CachePut`, and `@CacheEvict` to manage caching at the method level.

- `@Cacheable`: Caches the result of a method call.
- `@CachePut`: Updates the cache with the method result.
- `@CacheEvict`: Removes an entry from the cache

```
1  @Service
2  public class ProductService {
3      @Cacheable(value = "products", key = "#id")
4      public Product getProductById(Long id) {
5          return productRepository.findById(id).orElse(null);
6      }
7
8      @Cacheable(value = "products")
9      public List<Product> getAllProducts() {
10         return productRepository.findAll();
11     }
12
13     @CachePut(value = "products", key = "#product.id")
14     public Product createProduct(Product product) {
15         return productRepository.save(product);
16     }
17
18     @CachePut(value = "products", key = "#product.id")
19     public Product updateProduct(Product product) {
20         return productRepository.save(product);
21     }
22
23     @CacheEvict(value = "products", key = "#id")
24     public void deleteProduct(Long id) {
25         productRepository.deleteById(id);
26     }
27 }
```

```
9  @Configuration
10 public class CacheConfig {
11     @Bean
12     public RedisCacheManager cacheManager(RedisConnectionFactory
redisConnectionFactory) {
13         RedisCacheConfiguration cacheConfig =
RedisCacheConfiguration.defaultCacheConfig()
14
        .serializeKeysWith(RedisSerializationContext.SerializationPair.fromSerializer(new
StringRedisSerializer()));
15         return RedisCacheManager.builder(redisConnectionFactory)
16             .cacheDefaults(cacheConfig)
17             .build();
18     }
19 }
```

Caching with Custom Cache Manager



Best Practices for Caching